

A Django Basics

? What is Django?

Django is a high-level Python web framework that enables rapid development of secure and maintainable websites. It's free and open-source.

Key Characteristics:

- **Python-based:** Written in Python and leverages Python's clean syntax.
- **Batteries included:** Comes with a wide array of built-in features like user authentication, admin interface, ORM (Object Relational Mapper), form handling, URL routing, templating engine, etc.
- **Follows DRY Principle:** “Don’t Repeat Yourself.” Encourages reusability and clean, maintainable code.
- **Follows MVC Architecture** (more precisely, MTV — explained next).
- **Secure by default:** Protection against common security threats like SQL Injection, Cross-site scripting (XSS), Cross-site request forgery (CSRF), etc.

Use Django when you want to build:

- Content Management Systems (CMS)
- E-commerce websites
- APIs using Django REST Framework
- Blogs, Portals, Dashboards, or SaaS products

MTV Architecture (Model–Template–View)

Django follows an MTV architecture, which is a variation of MVC (Model–View–Controller).

Component	Role	Example
Model	Handles the data, database schema, ORM, and business logic	Python classes that define database tables
Template	Handles the presentation layer (HTML)	HTML files with Django Template Language (DTL)
View	Contains the logic that links Models and Templates	Python functions/classes that fetch data and render templates

In essence:

- Model = Database structure
- Template = HTML output
- View = Logic and control

Django handles the controller part behind the scenes (URL dispatcher).

☀ Why Use Django?

🔧 Features:

- Admin Panel: Auto-generated admin interface to manage data.
- ORM: Convert Python objects to database rows and vice versa.
- Authentication: Built-in login, logout, password hashing, sessions.
- Scalable: Handles large traffic when optimized properly (used by Instagram).
- Security: Mitigates XSS, CSRF, Clickjacking, SQL injection, etc.

- **Modular and Reusable:** Apps are self-contained and reusable.
- **REST Support:** Can be extended with Django REST Framework for APIs.

✓ Advantages:

- **Fast development with minimal code.**
 - **Well-documented and supported by a huge community.**
 - **Ideal for both beginner and expert Python developers.**
 - **Clean design promoting best practices.**
-

Setting Up a Django Project

📦 Installing Django with `pip`

Before installing Django, it's recommended to use a virtual environment to keep dependencies isolated.

bash

Copy code

Step 1: Create a virtual environment

```
python -m venv env
```

Step 2: Activate it

On Windows:

```
env\Scripts\activate
```

```
# On macOS/Linux:
```

```
source env/bin/activate
```

```
# Step 3: Install Django
```

```
pip install django
```

You can confirm the installation with:

```
bash
```

Copy code

```
django-admin --version
```



Creating a Project

Once Django is installed:

```
bash
```

Copy code

```
django-admin startproject myproject
```

This creates a new directory **myproject** with the following structure:

bash

Copy code

myproject/

├─ **manage.py**

└─ **myproject/**

 ├─ **__init__.py**

 ├─ **settings.py**

 ├─ **urls.py**

 ├─ **asgi.py**

 └─ **wsgi.py**

Project Structure Overview

File	Purpose
manage.py	Command-line utility to interact with the project (runserver, migrations, shell)
settings.py	Main configuration file: apps, databases, templates, middleware, etc.

urls.py **URL definitions for the project (root routing)**

asgi.py **Entry point for ASGI deployments (async support)**

wsgi.py **Entry point for WSGI deployments (traditional deployment)**

Django Apps

Creating an App

An App in Django is a modular component of your project. A project is a collection of apps.

To create an app:

bash

Copy code

```
python manage.py startapp blog
```

This creates a **blog** directory with:

bash

Copy code

```
blog/
```

```
|— __init__.py
```

- |— admin.py
- |— apps.py
- |— models.py
- |— tests.py
- |— views.py
- |— migrations/

File	Purpose
admin.py	Register models for Django admin interface
apps.py	Configuration of the app (used in INSTALLED_APPS)
models.py	Define your data models (tables)
views.py	Business logic (functions/classes) to handle HTTP requests
tests.py	Write unit tests
migrations/	Track schema changes over time

Aspect	Project	App
Purpose	Overall web application	Modular, self-contained components
Contains	Settings, global URLs, WSGI/ASGI config	Models, Views, Templates, URLs
Scope	Can include multiple apps	Focused on one functionality
Examples	E-commerce site	Cart app, Order app, User apps

You can plug an app into multiple projects, which makes Django apps reusable.

INSTALLED_APPS and Configuration

Inside `settings.py`, there's a list called **INSTALLED_APPS**:

python

Copy code

```
INSTALLED_APPS = [

    'django.contrib.admin',

    'django.contrib.auth',

    ...

    'blog',  # ← your custom app
```

]

What it does:

- Tells Django which apps are active for this project.
- Registers app configuration (defined in `apps.py`) with Django.
- Loads models from these apps.
- Includes app-specific middleware, signals, admin configuration, etc.

Adding your app:

Once you create an app (e.g., `blog`), you must add it here.

python

Copy code

```
# in settings.py
```

```
INSTALLED_APPS = [
```

```
    ...
```

```
    'blog.apps.BlogConfig', # recommended way
```

```
]
```

Why `'blog.apps.BlogConfig'`?

This refers to the `BlogConfig` class in `apps.py`:

python

Copy code

```
from django.apps import AppConfig
```

```
class BlogConfig(AppConfig):
```

```
    name = 'blog'
```

•

This approach is future-proof and supports advanced customization like signals and app-ready hooks.



Summary of App Lifecycle

1. Create the app: `python manage.py startapp appname`
2. Add to `INSTALLED_APPS`
3. Define models in `models.py`
4. Register models in `admin.py`
5. Define views in `views.py`
6. Map URLs in `urls.py` (you'll create this file in the app)
7. Create templates and static files
8. Run migrations: `makemigrations`, `migrate`
9. Run the server: `python manage.py runserver`



URL Dispatcher in Django — Master-Level Explanation

Django's URL Dispatcher is the system that routes HTTP requests to the correct view based on the URL in the browser.

Think of it as a traffic cop:

It matches the path of the request (e.g. `/blog/post/5`) to a specific view function that should process that request.

Why Is This Important?

- It's decoupled from the file system. URLs do not reflect directory structures.
 - You can define clean, semantic URLs like `/about/`, `/blog/post/5/`, etc.
 - You have complete control over URL behavior without changing the structure of your files or folders.
-

1. 📁 `urls.py` and URL Routing

When you create a Django project, Django sets up a root-level `urls.py` file, typically located in your project folder (the inner one).

Example file structure:

pgsql

Copy code

`myproject/`

|— `manage.py`

|— `myproject/` ← inner directory

|— `__init__.py`

|— `settings.py`

|— `urls.py` ← ROOT URL configuration

└─ wsgi.py

Initial `urls.py`:

python

Copy code

```
from django.contrib import admin
```

```
from django.urls import path
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
]
```

Let's break it down:

 `urlpatterns`

- It is a list that contains URL patterns.
- Each URL pattern maps a path to a view function or class.

 `path()`

Syntax:

python

Copy code

```
path('route/', view_function, name='optional_name')
```

Argument	Description
<code>'route/'</code>	The URL pattern. You can include dynamic variables here.
<code>view_function</code>	A function or class that handles the request
<code>name='xyz'</code>	An optional name that allows you to refer to this route elsewhere

Example:

python

Copy code

```
from django.http import HttpResponse
```

```
from django.urls import path
```

```
def home(request):
```

```
    return HttpResponse("Welcome to homepage!")
```

```
urlpatterns = [
```

```
    path('', home, name='home'),
```

]

Now:

- When a user visits `http://localhost:8000/`, Django calls the `home()` view.
-

2. 🧩 Including App URLs in Project URLs

When you build large projects with multiple apps (e.g., `blog`, `users`, `products`), each app should define its own `urls.py`. The project's `urls.py` then includes the app-level `urls.py`.

Why?

- Keeps your routing modular and organized
 - Apps become self-contained and reusable
-

🔧 Step-by-Step: Including App URLs

1 Create `urls.py` in your app (if not already present)

Suppose your app is `blog`. In `blog/urls.py`:

python

Copy code

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [

    path('', views.index, name='blog_home'),

    path('post/<int:id>/', views.post_detail, name='post_detail'),

]
```

2 Include this in your project-level `urls.py`

In `myproject/urls.py`:

python

Copy code

```
from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path('blog/', include('blog.urls')), # Forward anything starting
with /blog/ to blog app

]
```

How it Works:

URL Visited	Matched Pattern	View Called
-------------	-----------------	-------------

```
/blog/          '' from          views.index
                blog.urls
```

```
/blog/post/    'post/<int:id> views.post_detail(id
3/             /'          =3)
```

3. 📁 URL Patterns and Route Naming

Django encourages named URLs using the `name=` argument in `path()`.

🧠 Why Use Named Routes?

- Makes your code cleaner and safer: You can change the URL in one place without breaking every link.
 - Lets you reverse URLs in:
 - Templates (`{% url 'name' %}`)
 - Python code (`reverse('name')`)
 - Helps with DRY principle
-

📌 How to Define URL Names

In your app-level `urls.py`:

python

Copy code

```
urlpatterns = [
```

```
    path('', views.index, name='blog_home'),

    path('post/<int:id>/', views.post_detail, name='post_detail'),

]
```

Now your routes are:

- 'blog_home'
- 'post_detail'

Reversing URLs in Templates

html

Copy code

```
<!-- This generates /blog/ -->
```

```
<a href="{% url 'blog_home' %}">Home</a>
```

```
<!-- This generates /blog/post/3/ -->
```

```
<a href="{% url 'post_detail' id=3 %}">Read More</a>
```

No hardcoded paths like `/blog/post/3/`. Clean and maintainable!

Reversing URLs in Python Code

python

Copy code

```
from django.urls import reverse
```

```
from django.http import HttpResponseRedirect
```

```
def redirect_to_post():
```

```
    return HttpResponseRedirect(reverse('post_detail', args=[3]))
```

 URL Converters: `<type:name>`

Converter	Example	Description
<code>str</code> (default)	<code><str:name></code>	Matches any string excluding slashes
<code>int</code>	<code><int:id></code>	Matches 0 to 9999999999
<code>slug</code>	<code><slug:slug></code>	Matches letters, numbers, hyphens
<code>uuid</code>	<code><uuid:uid></code>	Matches a UUID

<code>path</code>	<code><path:subpath></code>	Matches any path including slashes
-------------------	-----------------------------------	------------------------------------

Example:

python

Copy code

```
path('post/<int:id>/', views.post_detail, name='post_detail')
```

This matches:

- `/post/3/` ✓
- `/post/hello/` ✗ (invalid for int)

✓ Summary: Mastering Django's URL Dispatcher

Topic	Summary
<code>urls.py</code>	Controls the routing logic
<code>path()</code>	Maps URLs to views
<code>include()</code>	Modularizes routing by app

`name=` Allows reverse lookup in templates and Python

`<int:id>` Dynamic segment to pass parameters to views

`{% url 'name' %}` Clean way to build links in templates

`reverse('name')` Clean way to generate URLs in Python code

✓ Real-World Example:

Imagine a blog site:

Project-level `urls.py`:

python

Copy code

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
urlpatterns = [
```

```
    path('admin/', admin.site.urls),
```

```
    path('blog/', include('blog.urls')),
```

```
]
```

App-level `urls.py`:

python

Copy code

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

```
    path('', views.index, name='blog_home'),
```

```
    path('post/<int:id>/', views.post_detail, name='post_detail'),
```

```
]
```

`views.py`:

python

Copy code

```
from django.shortcuts import render
```

```
def index(request):
```

```
return render(request, 'blog/index.html')
```

```
def post_detail(request, id):
```

```
    return render(request, 'blog/post_detail.html', {'post_id': id})
```

Template `index.html`:

html

Copy code

```
<h2>Blog Home</h2>
```

```
<a href="{% url 'post_detail' id=3 %}">Read Post 3</a>
```

❶ What Are Views in Django?

Views are the brain of a Django application.

They are Python functions (or classes) that:

- Receive HTTP requests
- Handle logic (e.g., fetch data, validate input)
- Return HTTP responses (HTML, JSON, redirects, errors, etc.)

🧠 Think of views as the bridge between your URLs and your templates/database.

❷ Function-Based Views (FBVs)

A Function-Based View (FBV) is a standard Python function with this signature:

python

Copy code

```
def view_name(request):  
  
    # your logic here  
  
    return HttpResponse("...")
```

Every view must accept a **request** parameter, which is an instance of **HttpRequest** provided by Django.

Basic Example

python

Copy code

```
from django.http import HttpResponse  
  
def hello(request):  
  
    return HttpResponse("Hello, Django!")
```

In **urls.py**:

python

Copy code

```
from django.urls import path
```

```
from .views import hello
```

```
urlpatterns = [  
    path('hello/', hello),  
]
```

Visiting `/hello/` now runs the `hello()` function and shows "Hello, Django!" in the browser.

FBV Logic Structure

Most FBVs follow this flow:

python

Copy code

```
def my_view(request):  
  
    # 1. Handle different request methods  
  
    # 2. Access request data (GET, POST, etc.)  
  
    # 3. Do something (query database, validate form, etc.)  
  
    # 4. Return a response (HTML, JSON, redirect)
```

③ Return Types in Views

1. **HttpResponse**

The most basic return type. You return a plain string of content (HTML/text).

python

Copy code

```
from django.http import HttpResponse
```

```
def homepage(request):  
  
    return HttpResponse("<h1>Welcome</h1>")
```

Common Parameters:

python

Copy code

```
HttpResponse(content, content_type='text/html', status=200)
```

Example:

python

Copy code

```
return HttpResponse("Not Found", status=404)
```

2. `render()`

The most common return type used when working with templates.

python

Copy code

```
from django.shortcuts import render
```

```
def home(request):  
  
    context = {'title': 'Welcome', 'user': 'Alice'}  
  
    return render(request, 'home.html', context)
```

What `render()` does:

- Loads `home.html` from the `templates/` folder
- Passes the `context` dictionary into the template
- Returns the HTML as an `HttpResponse`

Equivalent Manual Code:

python

Copy code

```
from django.template import loader

from django.http import HttpResponse


template = loader.get_template('home.html')

html = template.render(context, request)

return HttpResponse(html)
```

3. JsonResponse

Used to return JSON data, great for APIs or AJAX.

python

Copy code

```
from django.http import JsonResponse


def get_data(request):

    data = {'name': 'Alice', 'age': 30}

    return JsonResponse(data)
```

Output (as response):

json

Copy code

```
{  
  
    "name": "Alice",  
  
    "age": 30  
  
}
```

You can control behavior with:

python

Copy code

```
JsonResponse(data, safe=True, status=200, json_dumps_params={})
```

Note:

- `safe=True` means only dictionaries can be returned
- To return lists, set `safe=False`

4 Request and Response Objects



The `request` Object

Every view receives an `HttpRequest` object. It contains data about the current HTTP request.

Common **request** Attributes

Attribute	Purpose
<code>request.method</code>	"GET", "POST", etc.
<code>request.GET</code>	Query string data (e.g. <code>/search?q=test</code>)
<code>request.POST</code>	Form data (POST body)
<code>request.FILES</code>	Uploaded files
<code>request.COOKIES</code>	Cookies sent by browser
<code>request.session</code>	Django session object
<code>request.user</code>	Logged-in user (if <code>AuthenticationMiddleware</code> is used)
<code>request.path</code>	URL path (e.g. <code>/blog/</code>)
<code>request.META</code>	HTTP headers and environment variables

Example: Using GET and POST

python

Copy code

```
def search(request):  
  
    query = request.GET.get('q', '')  
  
    return HttpResponse(f"Results for: {query}")
```

For POST:

python

Copy code

```
def submit_form(request):  
  
    if request.method == 'POST':  
  
        name = request.POST['name']  
  
        return HttpResponse(f"Thanks, {name}")  
  
    return HttpResponse("Invalid method", status=405)
```

The response Object

You don't directly access a "response object", but you return one like:

- `HttpResponse(...)`
- `render(request, "template.html", context)`
- `JsonResponse({...})`
- `HttpResponseRedirect(...)`

You can customize responses:

python

Copy code

```
response = HttpResponse("Custom", status=201)

response['X-Special-Header'] = 'DjangoRocks'

return response
```

Setting Cookies in Response

python

Copy code

```
def set_cookie_view(request):

    response = HttpResponse("Cookie Set")

    response.set_cookie('mycookie', 'choco', max_age=3600)

    return response
```

Reading Cookies from Request

python

Copy code

```
def get_cookie_view(request):  
  
    flavor = request.COOKIES.get('mycookie', 'none')  
  
    return HttpResponse(f"Cookie: {flavor}")
```

Best Practices for Views

Practice	Why
Use <code>render()</code> for templates	Clean and readable
Use <code>JsonResponse</code> for APIs	Handles serialization, headers
Check request method (<code>GET/POST</code>)	Prevents unexpected form behavior
Use named URLs with <code>reverse()</code>	Flexible and maintainable
Use short, focused views	Easier to read and test

✓ Summary Table

Tool	Purpose
<code>HttpResponse()</code>	Return raw HTML/text
<code>JsonResponse()</code>	Return structured JSON data
<code>render()</code>	Combine templates and context
<code>request.method</code>	GET or POST logic
<code>request.GET, .POST</code>	Form and query data
<code>request.FILES</code>	File uploads
<code>request.user</code>	Access logged-in user
<code>request.COOKIES</code>	Read browser cookies
<code>response.set_cookie()</code>	Send cookies to client

1. Template Rendering with `render()`

In Django, templates are HTML files enhanced with Django Template Language (DTL). They help us dynamically generate HTML using data passed from views.

✓ How it works:

1. A view calls `render()`
2. Django loads a template file
3. It fills in any variables and template logic
4. Returns the final HTML as an `HttpResponse`

🔗 Example: A Simple View with Template

`views.py`

python

Copy code

```
from django.shortcuts import render
```

```
def home(request):
```

```
    context = {'name': 'Alice'}
```

```
    return render(request, 'home.html', context)
```

`home.html`

html

Copy code

```
<h1>Hello, {{ name }}</h1>
```

Output:

html

Copy code

```
<h1>Hello, Alice</h1>
```

📁 Where Do Templates Live?

By convention, you should create a **templates/** folder inside each app.

Copy code

```
myproject/
```

```
|
```

```
|— blog/
```

```
|   |— templates/
```

```
|   |   |— blog/
```

```
|   |       |— post_detail.html
```

To make sure Django finds your templates, add the following in **settings.py**:

python

Copy code

```
TEMPLATES = [  
  
    {  
  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
  
        'DIRS': [], # you can also add custom global template paths  
here  
  
        'APP_DIRS': True, # Important: enables per-app templates  
  
        ...  
  
    }  
  
]
```

2. Template Syntax

Django uses its own template language to embed logic inside HTML.

 `{{ variable }}`

Prints a variable passed from the context.

html

Copy code

```
<h2>Welcome, {{ user_name }}</h2>
```

Django will auto-escape HTML for safety:

python

Copy code

```
context = {'user_name': '<script>alert("XSS")</script>'}
```

Output will be:

html

Copy code

```
Welcome, &lt;script&gt;alert("XSS")&lt;/script&gt;
```

● {% block %}

Used in template inheritance. (More on this below)

html

Copy code

```
{% block content %}
```

```
<p>Override this content</p>
```

```
{% endblock %}
```

{% for %} Loop

Loop through a list or queryset.

html

Copy code

```
<ul>

    {% for item in items %}

        <li>{{ item }}</li>

    {% endfor %}

</ul>
```

You can use **forloop** variables:

html

Copy code

```
{{ forloop.counter }}      {# 1-based index #}

{{ forloop.counter0 }}     {# 0-based index #}

{{ forloop.first }}        {# True on first item #}

{{ forloop.last }}         {# True on last item #}
```

{% if %} Conditional

html

Copy code

```
{% if user.is_authenticated %}

    Hello, {{ user.username }}!

{% else %}

    Hello, guest!

{% endif %}
```

Also supports **elif** and **not**:

html

Copy code

```
{% if not user.is_staff %}

    You are not staff.

{% elif user.is_superuser %}

    You are a superuser.

{% endif %}
```

You can format output using filters:

html

Copy code

```
{{ name|lower }}           {# "alice" #}  
  
{{ price|floatformat:2 }} {# "19.99" #}  
  
{{ date|date:"Y-m-d" }}    {# "2025-05-21" #}
```

3. Template Inheritance (**base.html**)

Avoid repeating HTML structure like headers, footers, menus across every page.

Django uses a template inheritance system based on **extends** and **block**.

Step-by-Step

- ♦ base.html

html

Copy code

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
    <title>{% block title %}My Site{% endblock %}</title>
```

```
<link rel="stylesheet" href="{% static 'css/style.css' %}">

</head>

<body>

    <header>

        <h1>My Site</h1>

    </header>


    <main>

        {% block content %}

            Default content.

        {% endblock %}

    </main>


    <footer>© 2025</footer>

</body>

</html>
```

- ◆ **home.html (child)**

html

Copy code

```
{% extends 'base.html' %}

{% block title %}Home - My Site{% endblock %}


{% block content %}

    <p>This is the homepage!</p>

{% endblock %}
```

✓ This gives you clean, maintainable, DRY templates.

4. Static Files (CSS, JS, Images)

What Are Static Files?

These are files like:

- CSS stylesheets
- JavaScript files
- Images (logos, icons)

They are not dynamic — they don't change per request — but they are essential for frontend design.

Where to Put Static Files

Each app can have a **static/** folder:

cpp

Copy code

blog/

```
|— static/
|   |— blog/
|       |— css/
|           |— style.css
|           |— js/
|               |— script.js
```

Also define a global static folder (optional) for site-wide assets.

 **settings.py** Configuration

Make sure you have:

python

Copy code

```
STATIC_URL = '/static/'
```

(Optional for custom locations):

python

Copy code

```
STATICFILES_DIRS = [  
  
    BASE_DIR / "static",  
  
]
```

 Example: Using Static Files

1. Load static tag at top of template:

django

Copy code

```
{% load static %}
```

2. Use it to load files:

html

Copy code

```
<link rel="stylesheet" href="{% static 'blog/css/style.css' %}">
```

```

```

⚙️ `collectstatic` for Production

In development, static files are served automatically with `runserver`.

In production, run:

`bash`

Copy code

```
python manage.py collectstatic
```

This gathers all static files into `STATIC_ROOT` for deployment by your web server (e.g., Nginx).

✅ Summary: Django Templates Mastery

Concept	Summary
<code>render()</code>	Renders a template with context
<code>{{ variable }}</code>	Outputs values from context
<code>{% if %}</code> , <code>{% for %}</code>	Logic in templates
<code>{% block %}</code> , <code>{% extends %}</code>	For reusable base templates

`{% static %}`

For CSS/JS/image files

`static/` folder

Location for static assets

Filters (

`date,`

Real-World Scenario

Directory Structure:

`csharp`

Copy code

`myproject/`

`|— blog/`

`| |— templates/`

`| | |— blog/`

`| | |— base.html`

`| | |— post_list.html`

`| |— static/`

`| | |— blog/`

`| | |— css/style.css`

`views.py`

`python`

Copy code

```
def post_list(request):  
  
    posts = Post.objects.all()  
  
    return render(request, 'blog/post_list.html', {'posts': posts})
```

`post_list.html`

`html`

Copy code

```
{% extends 'blog/base.html' %}  
  
{% block content %}  
  
    <h2>All Posts</h2>  
  
    <ul>  
  
        {% for post in posts %}  
  
            <li>{{ post.title }}</li>  
  
        {% endfor %}  
  
    </ul>  
  
{% endblock %}
```

base.html

html

Copy code

```
<!DOCTYPE html>

<html>

<head>

    <title>{% block title %}Blog{% endblock %}</title>

    <link rel="stylesheet" href="{% static 'blog/css/style.css' %}">

</head>

<body>

    {% block content %}{% endblock %}

</body>

</html>
```

1. Models: Defining the Database Structure

What is a Django Model?

A Model is a Python class that represents a table in the database.

- Each attribute = a column
- Each instance = a row
- Defined by subclassing `models.Model`

Defining a Model

python

Copy code

```
from django.db import models

class Book(models.Model):

    title = models.CharField(max_length=200)

    description = models.TextField()

    published_date = models.DateField()

    is_published = models.BooleanField(default=False)
```

2. Fields in Models

Here are commonly used field types:

Field	Description
<code>CharField(max_length=...)</code>	Short text
<code>TextField()</code>	Long text

<code>IntegerField()</code>	Whole numbers
<code>BooleanField(default=...)</code>	True/False
<code>DateField()</code>	Date
<code>DateTimeField(auto_now_add=True)</code>	Date + time (on creation)
<code>EmailField()</code>	Validates email
<code>SlugField()</code>	For SEO-friendly URLs
<code>URLField()</code>	Validates a URL
<code>ImageField(upload_to=...)</code>	Image upload (needs Pillow)
<code>FileField()</code>	File upload

Special Options

- `blank=True`: field is optional in forms
- `null=True`: store **NULL** in DB
- `default=...`: default value

- `choices=[(..., ...)]`: restrict allowed values

python

Copy code

```
class Book(models.Model):

    GENRE_CHOICES = [

        ('fic', 'Fiction'),

        ('nfic', 'Non-fiction'),

    ]

    genre = models.CharField(max_length=10, choices=GENRE_CHOICES)
```

⚙️ 3. Meta Options

You define **Meta** as a nested class for model-level settings.

python

Copy code

```
class Book(models.Model):

    title = models.CharField(max_length=100)

    class Meta:
```

```
ordering = ['title'] # Default sort order
```

```
verbose_name = "Library Book"
```

Option	Purpose
<code>ordering</code>	Default order when querying
<code>verbose_name</code>	Human-readable name
<code>verbose_name_plural</code>	Plural version
<code>db_table</code>	Custom table name

4. Database Migrations

What Are Migrations?

Django tracks model changes and turns them into migration files. These files can be applied to the DB to keep it in sync.

Step-by-Step:

1. Create migration files:

bash

Copy code

```
python manage.py makemigrations
```

2. Apply migrations:

bash

Copy code

```
python manage.py migrate
```

3. View SQL that will be run:

bash

Copy code

```
python manage.py sqlmigrate app_name 0001
```

 Try Models in the Shell

bash

Copy code

```
python manage.py shell
```

python

Copy code

```
from yourapp.models import Book

Book.objects.create(title="Django", description="Deep Guide")

Book.objects.all()
```

Altering Models

Make a change in `models.py`, like adding a field:

python

Copy code

```
author = models.CharField(max_length=100, default="Unknown")
```

Then:

bash

Copy code

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Always provide `default=` or `null=True` when adding a non-nullable field to an existing model!

5. Django ORM Queries

Basic QuerySet Methods

Method	Description
<code>.all()</code>	All rows
<code>.filter(...)</code> <code>)</code>	Apply WHERE conditions
<code>.get(...)</code>	Get single object (raises error if 0 or >1 found)
<code>.exclude(...)</code> <code>.)</code>	Opposite of filter
<code>.count()</code>	Number of results

Examples:

python

Copy code

```
Book.objects.all()
```

```
Book.objects.filter(is_published=True)
```

```
Book.objects.get(id=1)
```

```
Book.objects.exclude(genre='fic')
```

```
Book.objects.count()
```

Complex Queries with Q Objects

python

Copy code

```
from django.db.models import Q
```

```
Book.objects.filter(  
    Q(title__icontains="django") | Q(description__icontains="python")  
)
```

Use **Q** for OR, AND, NOT logic.

Query Optimization

select_related

Used for foreign keys (1-to-1, many-to-1)

python

Copy code

```
Book.objects.select_related('author')
```

Reduces DB hits by doing JOINS.

```
prefetch_related
```

Used for many-to-many or reverse relations.

python

Copy code

```
Author.objects.prefetch_related('books')
```

Runs additional queries but still efficient (bulk loads related data).

👉 6. Model Relationships

🔗 ForeignKey

Defines a many-to-one relationship (many books to one author):

python

Copy code

```
class Author(models.Model):  
  
    name = models.CharField(max_length=100)
```

```
class Book(models.Model):
```

```
author = models.ForeignKey(Author, on_delete=models.CASCADE)
```

- `on_delete=models.CASCADE`: delete books if the author is deleted

OneToOneField

Used for 1-to-1 relationships (e.g. profile for each user):

python

Copy code

```
class Profile(models.Model):  
  
    user = models.OneToOneField(User, on_delete=models.CASCADE)
```

ManyToManyField

Used when both sides have multiple entries:

python

Copy code

```
class Book(models.Model):  
  
    title = models.CharField(max_length=100)  
  
    tags = models.ManyToManyField('Tag')
```

```
class Tag(models.Model):  
  
    name = models.CharField(max_length=50)
```

 **related_name**

Customize the reverse lookup name:

python

Copy code

```
class Book(models.Model):  
  
    author = models.ForeignKey(Author, on_delete=models.CASCADE,  
related_name='books')
```

Now:

python

Copy code

```
author = Author.objects.get(id=1)  
  
author.books.all() # instead of book_set.all()
```

 **Summary Table: Models & ORM Mastery**

Concept	Summary
<code>models.Model</code>	Base class for database tables
<code>Fields</code>	Define columns
<code>Meta</code>	Customize behavior (ordering, verbose_name)
<code>makemigrations / migrate</code>	Create & apply DB schema
<code>shell</code>	Interactively test queries
<code>.filter(), .get()</code>	Query the database
<code>Q()</code>	Complex queries
<code>select_related()</code>	Optimized JOIN for foreign keys
<code>prefetch_related()</code>	Efficient preloading for many-to-many
<code>ForeignKey, OneToOneField, ManyToManyField</code>	Define relationships
<code>related_name</code>	Customize reverse relation access

C. Forms & User Input in Django (Ultimate Guide)

Django's form system is powerful, secure, and highly extensible. It simplifies collecting, validating, and saving user input to the database.

Overview

There are two main types of forms in Django:

1. `forms.Form` — basic forms not tied to models
 2. `forms.ModelForm` — forms built from models (auto-generated fields + saving)
-

1. Django Forms

First, create a file: `forms.py` in your app directory.

A. Using `forms.Form` (manual fields)

python

Copy code

```
from django import forms
```

```
class ContactForm(forms.Form):
```

```
    name = forms.CharField(max_length=100)
```

```
    email = forms.EmailField()
```

```
    message = forms.CharField(widget=forms.Textarea)
```

B. Using `forms.ModelForm` (automated from models)

Example model:

python

Copy code

```
from django.db import models

class Comment(models.Model):

    name = models.CharField(max_length=100)

    email = models.EmailField()

    body = models.TextField()

    created_at = models.DateTimeField(auto_now_add=True)
```

Then create the form:

python

Copy code

```
from django import forms

from .models import Comment
```

```
class CommentForm(forms.ModelForm):  
  
    class Meta:  
  
        model = Comment  
  
        fields = ['name', 'email', 'body']
```

✓ 2. Validation Methods

A. `clean_<field>()`

Used to validate a specific field.

python

Copy code

```
class ContactForm(forms.Form):  
  
    email = forms.EmailField()  
  
  
    def clean_email(self):  
  
        email = self.cleaned_data['email']  
  
        if not email.endswith('.com'):  
  
            raise forms.ValidationError("Email must end with .com")  
  
        return email
```

B. `clean()`

Used for cross-field validation.

python

Copy code

```
class ContactForm(forms.Form):

    name = forms.CharField()

    email = forms.EmailField()

    def clean(self):

        cleaned_data = super().clean()

        name = cleaned_data.get('name')

        email = cleaned_data.get('email')

        if name and email and name.lower() in email:

            raise forms.ValidationError("Email should not contain your
name.")
```

3. Form Rendering in Templates

You can render forms in 3 ways:

A. `{{ form.as_p }}`

Quick and readable:

html

Copy code

```
<form method="post">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Send</button>

</form>
```

B. `{{ form.as_table }}` / `{{ form.as_ul }}`

Alternative formats.

C. Manual Rendering (full control)

html

Copy code

```
<form method="post">

    {% csrf_token %}

    <label for="id_name">Your Name</label>
```

```
{{ form.name }}
```

```
<label for="id_email">Email</label>
```

```
{{ form.email }}
```

```
<label for="id_message">Message</label>
```

```
{{ form.message }}
```

```
<button type="submit">Send</button>
```

```
</form>
```

You can access:

- `form.field_name.label`
- `form.field_name.errors`

4. Handling POST Requests in Views

Example View (function-based)

python

Copy code

```
from django.shortcuts import render, redirect

from .forms import ContactForm


def contact_view(request):

    if request.method == 'POST':

        form = ContactForm(request.POST)

        if form.is_valid():

            # Process data

            print(form.cleaned_data)

            return redirect('success_page')

    else:

        form = ContactForm()

    return render(request, 'contact.html', {'form': form})
```

CSRF Tokens (Mandatory)

In every form template:

html

Copy code

```
<form method="post">

    {% csrf_token %}
```

If missing, Django will block the request for security reasons.

5. Saving Data from a ModelForm

When using `ModelForm`, save data like this:

python

Copy code

```
def comment_view(request):

    if request.method == 'POST':

        form = CommentForm(request.POST)

        if form.is_valid():

            form.save()

            return redirect('post_detail')

    else:

        form = CommentForm()

    return render(request, 'comment_form.html', {'form': form})
```

You can also delay saving:

python

Copy code

```
comment = form.save(commit=False)

comment.post = current_post # set FK

comment.save()
```

❌ 6. Handling Form Errors

Django automatically includes field and form-wide errors.

Displaying them:

html

Copy code

```
<form method="post">

    {% csrf_token %}

    {{ form.non_field_errors }}

    {{ form.name.errors }}

    {{ form.name }}

    ...

</form>
```

Each field has its own `.errors`. The form has `non_field_errors`.

✓ 7. Messages Framework (Success/Failure Feedback)

Enable in `settings.py` (already enabled by default):

python

Copy code

```
INSTALLED_APPS = [  
  
    ...  
  
    'django.contrib.messages',  
  
]  
  
MIDDLEWARE = [  
  
    ...  
  
    'django.contrib.messages.middleware.MessageMiddleware',  
  
]
```

Use it in views:

python

Copy code

```
from django.contrib import messages

def contact_view(request):

    if request.method == 'POST':

        form = ContactForm(request.POST)

        if form.is_valid():

            # Do something

            messages.success(request, "Message sent successfully!")

            return redirect('contact')

        else:

            messages.error(request, "Something went wrong.")
```

Display messages in template:

html

Copy code

```
{% if messages %}

<ul class="messages">

    {% for message in messages %}

        <li class="{{ message.tags }}">{{ message }}</li>
```

```
        {% endfor %}

</ul>

{% endif %}
```

✓ Summary: Full Control Over Django Forms

Concept	Summary
<code>forms.Form</code>	Define fields manually
<code>forms.ModelForm</code>	Auto-generate fields from model
<code>clean_<field>()</code>	Field-specific validation
<code>clean()</code>	Cross-field validation
<code>{{ form.as_p }}</code>	Quick rendering
Manual HTML	Full control
CSRF token	Required for security
<code>.is_valid()</code>	Runs validation

`.cleaned_data`

Access validated data

`.save()`

Save form data to DB

`messages.success/error()`

Show user-friendly messages

Example Scenario: Contact Page with Custom Validation

- Validates `.com` emails
- Shows errors in HTML
- Sends success message on submit
- Posts data using `POST`
- Uses CSRF token

D. Authentication and Authorization in Django

Authentication and authorization are two **fundamental pillars of web security**. Django provides a **robust and extensible system** for handling both through its built-in `auth` app.

♦ 1. Built-in User Model

What is the User Model?

The **User model** is a representation of a user in the database. Django's default `User` model is provided by `django.contrib.auth.models.User`.

Fields of the Default User Model

python

CopyEdit

```
from django.contrib.auth.models import User
```

Core Fields:

- `username`: (Required, Unique) Primary identifier unless you're using a custom model.
- `password`: Hashed password (never stored in plain text).
- `email`: Email address (optional by default but can be made required).
- `first_name` / `last_name`: Optional fields for names.
- `is_staff`: Determines access to the Django admin site.
- `is_superuser`: Grants all permissions (even without explicitly assigning them).
- `is_active`: Soft-deletion flag. If `False`, login is disabled.
- `last_login`, `date_joined`: Useful timestamps.

Creating Users

Use the `User` manager:

python

CopyEdit

```
User.objects.create_user(username='john', password='secret')
```

It automatically **hashes the password** and saves the user. Do **not** set the password manually.

Creating Superusers

bash

CopyEdit

```
python manage.py createsuperuser
```

This prompts you for a username, email, and password. Superusers:

- Have `is_superuser = True` and `is_staff = True`
 - Can access the Django admin dashboard
 - Can do everything, bypassing object-level permission checks
-

♦ 2. Authentication Views

✅ What is Authentication?

Authentication means **verifying who the user is**. Django provides **views** for common actions like login, logout, and password management.

🔗 Built-in Views (Class-Based Views in `django.contrib.auth.views`)

🔒 LoginView

Handles login using a form (defaults to `AuthenticationForm`).

```
python
CopyEdit
from django.contrib.auth.views import LoginView
```

```
class MyLoginView(LoginView):
    template_name = 'login.html'
```

- **GET**: Renders the login form
- **POST**: Authenticates and logs in user if credentials are valid

🔒 LogoutView

Logs the user out and can redirect to another page.

```
python
```

CopyEdit

```
from django.contrib.auth.views import LogoutView
```

```
class MyLogoutView(LogoutView):  
    next_page = '/login/' # Optional
```

PasswordResetView

Initiates the password reset process by sending an email with a reset link.

Other related views:

- PasswordResetConfirmView
- PasswordResetDoneView
- PasswordResetCompleteView

♦ 3. Custom Login/Signup Forms

Django allows you to create **your own forms** using `forms.Form` or `forms.ModelForm`.

Example: Custom Login Form

python

CopyEdit

```
from django import forms
```

```
class CustomLoginForm(forms.Form):  
    username = forms.CharField()  
    password = forms.CharField(widget=forms.PasswordInput)
```

Use it with your own login view or override the form used by `LoginView`:

python

CopyEdit

```
class MyLoginView(LoginView):  
    form_class = CustomLoginForm
```

```
template_name = 'login.html'
```

Example: Custom Signup Form

python

CopyEdit

```
from django.contrib.auth.models import User
from django import forms

class SignupForm(forms.ModelForm):
    password = forms.CharField(widget=forms.PasswordInput)

    class Meta:
        model = User
        fields = ['username', 'email', 'password']

    def save(self, commit=True):
        user = super().save(commit=False)
        user.set_password(self.cleaned_data['password']) # Hashes
password
        if commit:
            user.save()
        return user
```

4. @login_required Decorator

Purpose

This decorator ensures the view is **accessible only to authenticated users**.

Usage

python

CopyEdit

```
from django.contrib.auth.decorators import login_required

@login_required
def my_view(request):
    return render(request, 'protected.html')
```

- If user is **not logged in**, they're redirected to `LOGIN_URL` (default: `/accounts/login/`)
 - Can be applied to function-based views (FBVs)
 - For class-based views, use `LoginRequiredMixin`
-

5. Permissions

Django Permission System

Permissions control **what an authenticated user is allowed to do**.

Built-in Permissions

Django creates **3 permissions per model** by default:

- `add_modelname`
- `change_modelname`
- `delete_modelname`

Checking Permissions

```
python
```

```
CopyEdit
```

```
user.has_perm('app_label.permission_codename')
```

Key User Flags

`is_staff`

- Can access the Django admin site if `True`

- Does **not grant model permissions** by itself

is_superuser

- Bypasses all permission checks
 - **Has every permission** by default
 - Should be used sparingly
-

Object-Level Permissions

What Are They?

Permissions **applied to specific objects**, not just models. For example, allowing a user to edit only their own blog post.

Django by default doesn't support this natively out of the box. You'll need:

- Custom logic in views
 - OR use third-party apps like **django-guardian**
-

PermissionRequiredMixin

This mixin restricts access to **class-based views** unless the user has specific permissions.

Usage

python

CopyEdit

```
from django.contrib.auth.mixins import PermissionRequiredMixin
from django.views.generic import View
```

```
class MyView(PermissionRequiredMixin, View):
    permission_required = 'app_name.change_modelname'
```

```
raise_exception = True # Raise 403 instead of redirecting to login
```

- `permission_required`: Can be a single permission or a list
 - Mix it in **before** the view class (`PermissionRequiredMixin`, `MyView`)
-

Summary Table

Feature	Function/Use
User model	Represents a user
<code>create_user()</code>	Safely creates a normal user
<code>createsuperuser</code>	CLI to create superusers
<code>LoginView</code>	View to log in users
<code>LogoutView</code>	View to log out users
<code>PasswordResetView</code>	Starts password reset
<code>@login_required</code>	Restrict FBVs to authenticated users
<code>is_staff</code>	Can access admin
<code>is_superuser</code>	Has all permissions
<code>has_perm()</code>	Check user's permission
<code>PermissionRequiredMixin</code>	Restrict CBVs to permission-holding users
Object permissions	Allow per-object access control

What You Should Implement in Your App

- Use `@login_required` or `LoginRequiredMixin` on sensitive views

- Always use `set_password()` when storing passwords
- Customize your login and signup forms for better UX
- Use `PermissionRequiredMixin` for admin/restricted actions
- Implement object-level permissions for fine-grained access
- Never hardcode `is_superuser` checks unless absolutely necessary

E. Class-Based Views (CBVs) in Django

What Are Class-Based Views?

CBVs are Django views implemented as **Python classes** rather than functions (as in FBVs). They offer:

- **Code reuse through inheritance**
- **Clear separation of concerns** (e.g., data handling, context preparation, form validation)
- **Ease in extending default behavior** via overriding methods

CBVs use the **object-oriented paradigm** to make views more flexible and modular.

♦ 1. Generic CBVs (The Power Tools)

Django provides a set of **Generic CBVs** that implement common patterns like listing objects, viewing details, creating/updating/deleting objects.

You can find them in:

```
python
CopyEdit
from django.views.generic import ListView, DetailView, CreateView,
UpdateView, DeleteView
```

ListView

Used to **display a list of objects**.

python

CopyEdit

```
class ArticleListView(ListView):
    model = Article
    template_name = 'articles/list.html'
    context_object_name = 'articles'
    paginate_by = 10
```

Key Attributes:

- `model`: The model to fetch data from
- `context_object_name`: The name used in the template (default: `object_list`)
- `paginate_by`: Automatically handles pagination

Template Context:

- `object_list`: List of objects
- `page_obj`, `paginator`: When using pagination

DetailView

Displays **details of a single object**.

python

CopyEdit

```
class ArticleDetailView(DetailView):
    model = Article
    template_name = 'articles/detail.html'
    context_object_name = 'article'
```

How it finds the object:

- Uses `pk` or `slug` from the URL
- URL pattern must include `pk` or `slug` parameter

```
python
CopyEdit
path('article/<int:pk>/', ArticleDetailView.as_view(),
name='article-detail')
```

CreateView

Displays a form to **create a new object**, and saves it upon POST.

```
python
CopyEdit
class ArticleCreateView(CreateView):
    model = Article
    fields = ['title', 'content']
    template_name = 'articles/form.html'
    success_url = '/articles/'
```

Internally uses:

- `get_form()`, `form_valid()`, `form_invalid()`
-

UpdateView

Same as `CreateView`, but pre-populates the form with existing object data.

```
python
CopyEdit
class ArticleUpdateView(UpdateView):
    model = Article
    fields = ['title', 'content']
    template_name = 'articles/form.html'
    success_url = '/articles/'
```

URL must provide a primary key or slug.

DeleteView

Asks for confirmation, then **deletes the object**.

```
python
CopyEdit
class ArticleDeleteView(DeleteView):
    model = Article
    template_name = 'articles/confirm_delete.html'
    success_url = '/articles/'
```

Requires a confirmation template.

2. URL Mapping with `as_view()`

`as_view()` turns a class into a view function that Django understands.

Example:

```
python
CopyEdit
from django.urls import path
from .views import ArticleListView

urlpatterns = [
    path('articles/', ArticleListView.as_view(), name='article-list'),
]
```

Why `as_view()`?

- It instantiates the class and calls the appropriate method like `get()` or `post()` depending on HTTP request.

⚙️ 3. Customizing CBVs

You can **override methods** to inject custom logic. The most useful ones:

🔧 Overriding `get_context_data()`

Used to **add extra data** to the context sent to the template.

```
python
CopyEdit
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context['author'] = self.request.user
    return context
```

- `super().get_context_data(**kwargs)` fetches the default context (like `object_list` or `form`)
- You can add anything—additional objects, forms, strings, booleans

🧠 Overriding `form_valid(form)`

Called when a POST form is **successfully validated**.

```
python
CopyEdit
def form_valid(self, form):
    form.instance.author = self.request.user
    return super().form_valid(form)
```

- You can **inject data** before saving (e.g., current user)
- Always return `super().form_valid(form)` unless you're doing something completely custom

4. Mixing Function-Based Views (FBVs) and CBVs

You **can mix** FBVs and CBVs in the same project/app. It's about **choosing the right tool** for the right task.

When to Use CBVs:

- CRUD operations
- Shared view logic
- Need to extend or subclass views

When to Use FBVs:

- Simple views with few lines
- Custom behavior that doesn't fit cleanly into CBVs
- Views with many conditionals or early returns

Example FBV + CBV in one app:

python

CopyEdit

views.py

```
def homepage(request):  
    return render(request, 'homepage.html')
```

```
class BlogListView(ListView):  
    model = Blog
```

python

CopyEdit

urls.py

```
path('', views.homepage),  
path('blogs/', views.BlogListView.as_view()),
```

🧩 5. Using Mixins

Mixins are reusable pieces of behavior you can combine with CBVs. They are Python classes **designed to be inherited** with CBVs.

✅ Common Mixins

🔑 LoginRequiredMixin

Used to restrict CBVs to authenticated users.

```
python
CopyEdit
from django.contrib.auth.mixins import LoginRequiredMixin

class DashboardView(LoginRequiredMixin, TemplateView):
    template_name = 'dashboard.html'
```

✅ Must be placed **before** the view class due to Python's MRO (Method Resolution Order)

🔑 PermissionRequiredMixin

Restricts access based on permissions.

```
python
CopyEdit
from django.contrib.auth.mixins import PermissionRequiredMixin

class EditPostView(PermissionRequiredMixin, UpdateView):
    permission_required = 'blog.change_post'
    model = Post
    fields = ['title', 'body']
    template_name = 'edit.html'
    success_url = '/posts/'
```

🔧 Custom Mixins

You can create your own mixins for **reusable logic**.

```
python
CopyEdit
class StaffRequiredMixin:
    def dispatch(self, request, *args, **kwargs):
        if not request.user.is_staff:
            return redirect('not-authorized')
        return super().dispatch(request, *args, **kwargs)
```

Use it like:

```
python
CopyEdit
class AdminView(StaffRequiredMixin, ListView):
    model = Report
    template_name = 'admin/reports.html'
```

Summary Table

CBV Type	Purpose	Methods to Override
ListView	Display list of objects	get_queryset, get_context_data
DetailView	Show object detail	get_object, get_context_data
CreateView	Create new objects	form_valid, get_context_data
UpdateView	Update existing objects	form_valid, get_object
DeleteView	Delete objects	get_object, delete
LoginRequiredMixin	Restrict access	dispatch()
PermissionRequiredMixin	Check permissions	has_permission()

Best Practices

- Use **CBVs for CRUD** and repeatable patterns.
 - Customize using `get_context_data`, `form_valid`, etc.
 - Use `LoginRequiredMixin` to protect views.
 - Always call `super()` when overriding methods.
 - Know when to drop to FBVs for simplicity.
-

Conclusion

Django CBVs give you a **powerful, reusable, and organized** approach to writing views. By mastering them and combining them with mixins and method overrides, you can:

- Build clean, scalable views
- Avoid duplicated code
- Maintain better separation of concerns

F. Templates in Depth

Templates are the **presentation layer** in Django's MTV (Model-Template-View) architecture. They're written using the **Django Template Language (DTL)** and serve the role of generating **dynamic HTML**.

♦ 1. Django Template Filters and Tags

A. What Are Template Filters?

Filters transform the values of variables in templates.

Syntax:

django

CopyEdit
{{ value|filter_name }}

Example:

django
CopyEdit
{{ post.published_date|date:"F j, Y" }}

This will render something like:

yaml
CopyEdit
March 5, 2025

B. Built-in Filters

Django comes with **dozens** of built-in filters. Here are the most important ones:

date

Formats a `datetime` object.

django
CopyEdit
{{ object.created_at|date:"Y-m-d" }} → 2025-05-21

Common format strings:

- `Y`: Year with century
- `m`: Month (01-12)
- `d`: Day (01-31)
- `F`: Full month name

- **j**: Day (no leading zero)

length

Returns the **length** of a list, string, queryset, etc.

django

CopyEdit

```
{{ blog_posts|length }} → 4
```

truncatewords

Limits a string to **N words**.

django

CopyEdit

```
{{ post.content|truncatewords:20 }}
```

Useful for previews or summaries.

More Useful Filters

Filter	Purpose
lower	Converts text to lowercase
upper	Converts text to uppercase
title	Capitalizes each word
default	Uses a fallback value
safe	Marks HTML as safe (renders HTML tags)
linebreaks	Converts <code>\n</code> into <code><p></code> and <code>
</code>
join	Joins a list into a string

C. Template Tags

Tags control **logic and flow** in templates.

Syntax:

django
CopyEdit
`{% tag_name arguments %}`

{% for %}

Loops over lists/querysets.

django
CopyEdit
`{% for post in posts %}`
 `<h2>{{ post.title }}</h2>`
`{% endfor %}`

{% if %}

Used for conditionals.

django
CopyEdit
`{% if user.is_authenticated %}`
 `Welcome {{ user.username }}!`
`{% else %}`
 `Please log in.`
`{% endif %}`

{% url %}

Generates a URL from the URL name in `urls.py`.

django

CopyEdit

```
<a href="{% url 'blog:detail' post.pk %}">Read more</a>
```

🔧 {% block %} & {% extends %}

Used in **template inheritance**.

django

CopyEdit

```
<!-- base.html -->
<html>
  <body>
    {% block content %}{% endblock %}
  </body>
</html>

<!-- home.html -->
{% extends 'base.html' %}
{% block content %}
  <h1>Home Page</h1>
{% endblock %}
```

📦 D. Custom Template Filters and Tags

If built-in filters/tags aren't enough, you can define your own.

✅ 1. Create a Template Tags File

In any app:

bash

CopyEdit

```
your_app/
├── templatetags/
│   ├── __init__.py
│   └── custom_tags.py
```

✓ 2. Define a Custom Filter

python

CopyEdit

```
# your_app/templatetags/custom_tags.py
from django import template
```

```
register = template.Library()
```

```
@register.filter
```

```
def shout(value):
    return value.upper() + "!"
```

✓ 3. Use It in Templates

django

CopyEdit

```
{% load custom_tags %}
{{ "hello"|shout }} → HELLO!
```

✓ 4. Define a Custom Tag

Tags are more complex than filters.

python

CopyEdit

```
@register.simple_tag
```

```
def multiply(a, b):
    return a * b
```

django

CopyEdit

```
{% load custom_tags %}
{% multiply 5 3 %} → 15
```

2. Static & Media Files

A. What Are Static Files?

Static files are assets like:

- CSS
- JavaScript
- Images that don't change

They are part of the **frontend**, not user-uploaded content.

Static File Settings

python

CopyEdit

```
# settings.py
```

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / "static", # development files  
]
```

```
STATIC_ROOT = BASE_DIR / 'staticfiles' # for collectstatic
```

Using Static Files in Templates

1. Add `{% load static %}` at the top.
2. Use `{% static 'path/to/file.css' %}`.

django

CopyEdit

```
{% load static %}
```

```
<link rel="stylesheet" href="{% static 'css/styles.css' %}">
```

```

```

collectstatic in Production

Use this before deployment:

```
bash
CopyEdit
python manage.py collectstatic
```

It gathers all static files into `STATIC_ROOT`.

B. Media Files (User-uploaded files)

Media files are things like:

- Uploaded images
- Videos
- Documents

They change based on user input.

Media Settings

```
python
CopyEdit
# settings.py
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Handling Uploads in Models

```
python
CopyEdit
from django.db import models
```

```
class Profile(models.Model):
    name = models.CharField(max_length=50)
    avatar = models.ImageField(upload_to='avatars/')
```

This will store uploads in:

```
bash
CopyEdit
media/avatars/filename.jpg
```

Serve Media Files in Development

```
python
CopyEdit
# urls.py (development only)
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # ... your URLs
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

! Never serve media via Django in production. Use Amazon S3, Cloudinary, etc.

Displaying Uploaded Images in Templates

```
django
CopyEdit

```

Make sure you:

Have `Pillow` installed for `ImageField`:

```
bash
CopyEdit
pip install Pillow
```

-

🧠 Summary Table

Feature	Purpose	Usage/Example
<code>{{ var filter }}</code>		Apply transformation to variable
<code>{% if %}</code>	Conditional rendering	<code>{% if user.is_authenticated %}</code>
<code>{% for %}</code>	Loop over data	<code>{% for item in list %}</code>
<code>{% url %}</code>	Generate URL from view name	<code>{% url 'post-detail' post.pk %}</code>
<code>{% static %}</code>	Load static assets	<code>{% static 'css/style.css' %}</code>
<code>{% load custom_tags %}</code>	Load custom filters/tags	<code>{% multiply 5 2 %}</code>
<code>MEDIA_URL</code>	URL prefix for user-uploaded files	<code>/media/avatars/pic.jpg</code>
<code>STATICFILES_DIRS</code>	Where dev static files live	<code>[BASE_DIR / "static"]</code>
<code>STATIC_ROOT</code>	Collectstatic output for production	<code>BASE_DIR / "staticfiles"</code>

✅ Best Practices

- Organize static files per app (e.g., `app/static/app/style.css`)
- Use custom filters for formatting logic (e.g., currency formatting)
- Serve media files with caution—never directly in production
- Load only required static files for performance
- Use `{% block %}` and `{% extends %}` to avoid repetition

Want a Project Example?

Let me know if you want a **mini-project** combining:

G. Testing in Django

Testing is a **crucial step** in software development to ensure your code behaves as expected.

1. Django's Built-In Testing Framework

Django's testing framework is built on Python's built-in `unittest` module, but adds Django-specific utilities.

Test Discovery

Django automatically finds tests in:

- Any file named `test*.py` in your app
- In test case classes that inherit from `django.test.TestCase`

To run all tests:

```
bash
CopyEdit
python manage.py test
```

2. Django Test Client

The **test client** is a Python class that simulates a dummy web browser. It lets you test views and URLs.

Setup

```
python
CopyEdit
from django.test import TestCase
```

```
class MyTests(TestCase):
    def test_homepage_status_code(self):
        response = self.client.get('/')
        self.assertEqual(response.status_code, 200)
```

Test Methods

Method	Description
<code>get(url)</code>	Simulates GET request
<code>post(url, data)</code>	Simulates POST with data
<code>put, delete</code>	Simulate other HTTP verbs
<code>login()</code>	Simulate user login
<code>logout()</code>	Simulate user logout

Example with Login

```
python
CopyEdit
def test_login(self):
    self.client.post('/login/', {'username': 'admin', 'password':
'123'})
    response = self.client.get('/dashboard/')
    self.assertEqual(response.status_code, 200)
```

3. Writing Unit Tests for Views, Models, and Forms

A. Model Tests

Model tests ensure that **business logic and data constraints** are correct.

```
python
CopyEdit
from django.test import TestCase
```

```
from .models import Post

class PostModelTest(TestCase):
    def test_post_title(self):
        post = Post.objects.create(title='Test Title',
content='Sample')
        self.assertEqual(post.title, 'Test Title')
```

B. View Tests

Used with the **test client**, they verify:

- HTTP status codes
- Template rendering
- Redirects
- Context data

python

CopyEdit

```
def test_post_detail_view(self):
    post = Post.objects.create(title='Test', content='Text')
    response = self.client.get(f'/posts/{post.id}/')
    self.assertEqual(response.status_code, 200)
    self.assertTemplateUsed(response, 'post_detail.html')
```

C. Form Tests

Ensure that forms:

- Accept valid data
- Reject invalid data
- Validate fields properly

```
python
CopyEdit
from .forms import PostForm

class PostFormTest(TestCase):
    def test_valid_form(self):
        form = PostForm(data={'title': 'Test', 'content': 'Content'})
        self.assertTrue(form.is_valid())

    def test_invalid_form(self):
        form = PostForm(data={'title': ''})
        self.assertFalse(form.is_valid())
```

⚡ 4. Using **pytest** with Django

pytest is a powerful and user-friendly alternative to **unittest**.

✅ Installation

```
bash
CopyEdit
pip install pytest pytest-django
```

Add to **pytest.ini**:

```
ini
CopyEdit
[pytest]
DJANGO_SETTINGS_MODULE = your_project.settings
```

🔧 Example Test with Pytest

```
python
CopyEdit
import pytest
from django.urls import reverse
from .models import Post
```

```
@pytest.mark.django_db
def test_post_list_view(client):
    Post.objects.create(title="Hello", content="World")
    response = client.get(reverse("post-list"))
    assert response.status_code == 200
    assert "Hello" in response.content.decode()
```

- `client` is a pytest fixture, equivalent to Django's test client.
- `@pytest.mark.django_db` is required when accessing the database.

Benefits of Pytest

Feature	Benefit
Fewer imports	No need to inherit from <code>TestCase</code>
Fixtures	Easy reusable test setup
Rich assertion syntax	<code>assert x == y</code> is much simpler

Debugging in Django

1. Django Debug Toolbar

The **Django Debug Toolbar** is a web-based panel that shows:

- SQL queries
- Request/response data
- Templates used
- Cache usage
- Signals

Installation

bash

CopyEdit

```
pip install django-debug-toolbar
```

Setup in `settings.py`

python

CopyEdit

```
INSTALLED_APPS += ['debug_toolbar']
```

```
MIDDLEWARE = ['debug_toolbar.middleware.DebugToolbarMiddleware'] +  
MIDDLEWARE
```

```
INTERNAL_IPS = ['127.0.0.1'] # for local development
```

Setup in `urls.py`

python

CopyEdit

```
from django.conf import settings  
from django.urls import include
```

```
if settings.DEBUG:  
    import debug_toolbar  
    urlpatterns = [  
        path('__debug__/', include(debug_toolbar.urls)),  
    ] + urlpatterns
```

What You Can See

- SQL panel: Shows how many queries were run and their execution time
- Templates panel: Which templates were rendered and their inheritance
- Timer: Total processing time
- Headers: Request and response headers

- Static files panel, Cache panel, etc.

Example Use Case

If a view is running **100+ SQL queries**, Debug Toolbar will show that, helping you optimize with `select_related()` and `prefetch_related()`.

2. Custom Error Pages (404 and 500)

You can create custom error pages to improve the user experience.

Custom 404 Page

1. Create `404.html` in your templates directory.
2. Set `DEBUG = False` and add proper `ALLOWED_HOSTS` in `settings.py`.

html

CopyEdit

```
<!-- templates/404.html -->
<h1>Oops! Page Not Found.</h1>
<p>The page you're looking for does not exist.</p>
```

Custom 500 Page

1. Create `500.html` in templates.
2. Ensure Django can serve it when an internal error occurs.

html

CopyEdit

```
<!-- templates/500.html -->
<h1>Internal Server Error</h1>
<p>Something went wrong on our side.</p>
```

⚠ Important Notes

- Custom error pages only work when `DEBUG = False`.
- Make sure `ALLOWED_HOSTS` is set, or Django won't serve any request.

🔍 Testing Error Pages

You can test error views manually:

```
python
CopyEdit
from django.http import HttpResponseServerError

def trigger_error(request):
    return HttpResponseServerError("Internal Error")
```

Map it in `urls.py` to simulate `500`.

🧠 Summary Table

Topic	What It Does	Tool/API
Django Test Client	Simulates requests	<code>self.client.get/post()</code>
Unit Tests	Test models, views, forms	<code>TestCase</code> , <code>assertEqual</code>
<code>pytest</code>	Advanced testing framework	<code>pytest.mark.django_db</code>
Django Debug Toolbar	Debugging panel for dev	<code>__debug__</code> / route
Custom 404/500 Pages	User-friendly error handling	<code>404.html</code> , <code>500.html</code>

Want Hands-On Practice?

I can walk you through:

- Writing pytest-based tests for a blog
- Measuring SQL queries with Debug Toolbar
- Triggering custom error pages in development

Just say the word.

Do you want to try that now?

40

H. Advanced Django Topics

1. Advanced ORM

Django's ORM (Object-Relational Mapper) becomes truly powerful when you go beyond basic queries and start leveraging **annotations**, **aggregations**, **expressions**, and **raw SQL**.

1.1 **annotate()** – Add Calculated Fields to Querysets

Use **annotate()** to **add new fields to each object** in a queryset, calculated from other fields or related objects.

Example: Count Comments for Each Post

python

CopyEdit

```
from django.db.models import Count
Post.objects.annotate(num_comments=Count('comments'))
```

Each **Post** object now has **.num_comments**.

1.2 `aggregate()` – Summarize a Whole Queryset

Use `aggregate()` to return a **single dictionary** summarizing the entire queryset.

✓ Example: Get the Average Price of Products

python

CopyEdit

```
from django.db.models import Avg
Product.objects.aggregate(avg_price=Avg('price'))
```

Returns:

python

CopyEdit

```
{'avg_price': 125.50}
```

1.3 `F()` Expressions – Avoid Python Evaluation

Use `F()` expressions to refer to **model field values on the database side**, allowing you to update without retrieving first.

✓ Example: Increment `view_count` by 1

python

CopyEdit

```
from django.db.models import F
Post.objects.update(view_count=F('view_count') + 1)
```

This translates directly into SQL: `UPDATE post SET view_count = view_count + 1.`

1.4 Complex Example

python

CopyEdit

```
from django.db.models import Count, Avg, F

Post.objects.filter(published=True)\
    .annotate(comment_count=Count('comments'))\
    .aggregate(avg_comments=Avg('comment_count'))
```

2. 🖥️ Raw SQL Queries

When the ORM can't handle a complex query, you can use raw SQL safely.

python

CopyEdit

```
Post.objects.raw('SELECT * FROM blog_post WHERE title = %s',  
['Django'])
```

- Returns a queryset of `Post` instances
- Placeholders (`%s`) are database-agnostic and protect from SQL injection

You can also use `connection.cursor()` if you want lower-level access:

python

CopyEdit

```
from django.db import connection
```

```
with connection.cursor() as cursor:  
    cursor.execute("SELECT COUNT(*) FROM blog_post")  
    row = cursor.fetchone()
```

3. 🛎️ Django Signals

Signals let you respond to **specific events** (e.g., model save/delete) without tightly coupling logic.

3.1 Pre-save and Post-save

- `pre_save`: Before saving the model to DB
- `post_save`: After saving

```
python
CopyEdit
from django.db.models.signals import pre_save, post_save
from django.dispatch import receiver
from .models import Post

@receiver(post_save, sender=Post)
def notify_author(sender, instance, created, **kwargs):
    if created:
        print(f"New post titled '{instance.title}' has been created!")
```

3.2 Signal Parts

Component	Description
<code>@receiver</code>	Decorator to connect the signal
<code>sender</code>	The model that triggers the signal
<code>instance</code>	The actual object being saved
<code>created</code>	Boolean (only <code>post_save</code>) for new objects

You can also **manually connect** signals without decorators:

```
python
CopyEdit
post_save.connect(notify_author, sender=Post)
```

4. Custom User Models

4.1 Why Customize?

Default `User` includes `username`, `email`, `password`, but lacks flexibility. You may want to:

- Use email instead of username

- Add fields like `phone_number` or `profile_image`
-

4.2 Extending AbstractUser

python

CopyEdit

```
from django.contrib.auth.models import AbstractUser
from django.db import models
```

```
class CustomUser(AbstractUser):
    phone_number = models.CharField(max_length=15)
```

4.3 AUTH_USER_MODEL Setting

In `settings.py`:

python

CopyEdit

```
AUTH_USER_MODEL = 'myapp.CustomUser'
```

Important:

- Set this before running `migrate` the first time.
- Reference user model via `settings.AUTH_USER_MODEL`.

4.4 Create User via Manager

python

CopyEdit

```
from django.contrib.auth import get_user_model
User = get_user_model()
User.objects.create_user(username='test', password='pass')
```

5. File Uploads

5.1 FileField and ImageField

python

CopyEdit

```
class Document(models.Model):
    file = models.FileField(upload_to='docs/')
    image = models.ImageField(upload_to='images/')
```

- `upload_to` defines the folder inside `MEDIA_ROOT`.

5.2 Media Settings

In `settings.py`:

python

CopyEdit

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

In `urls.py` (for development):

python

CopyEdit

```
from django.conf import settings
from django.conf.urls.static import static
```

```
urlpatterns += static(settings.MEDIA_URL,
                      document_root=settings.MEDIA_ROOT)
```

5.3 Handling Uploads in Views

python

CopyEdit

```
def upload_file(request):
    if request.method == 'POST':
```

```
form = UploadForm(request.POST, request.FILES)
if form.is_valid():
    form.save()
```

Use `request.FILES['file_field_name']`.

5.4 Security Best Practices

- Validate file types: only allow `.jpg`, `.pdf`, etc.
 - Use `ImageField` to auto-verify image MIME
 - Store sensitive files outside `MEDIA_ROOT` and access via secure views
 - Don't trust file names — Django renames with `upload_to` + a hash/suffix
-

6. 🧩 Django Middleware

6.1 What is Middleware?

Middleware is a **hook layer** between the request and response. It can:

- Process requests/responses globally
- Modify request objects before views
- Handle exceptions, redirects, etc.

Django's Built-in Middleware

Some examples:

Middleware

Purpose

<code>SecurityMiddleware</code>	Forces HTTPS, security headers
<code>SessionMiddleware</code>	Enables sessions
<code>AuthenticationMiddleware</code>	Sets <code>request.user</code> are
<code>CommonMiddleware</code>	Adds trailing slashes, etc.

6.2 Creating Custom Middleware

Custom middleware is a **Python class with special methods**.

✓ Basic Template

python

CopyEdit

```
class MyCustomMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print("Before view")
        response = self.get_response(request)
        print("After view")
        return response
```

Add to `settings.py`:

python

CopyEdit

```
MIDDLEWARE = [
    'myapp.middleware.MyCustomMiddleware',
    ...
]
```

Advanced Middleware Methods

python

CopyEdit

```
class LoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        return self.get_response(request)

    def process_view(self, request, view_func, view_args,
view_kwargs):
        print(f"Calling view: {view_func.__name__}")

    def process_exception(self, request, exception):
        print(f"Exception occurred: {exception}")
```

Summary Table

Concept	Key Feature	Use Case Example
<code>annotate()</code>	Add field to each object	Count comments for each post
<code>aggregate()</code>	Return summary for entire queryset	Avg price of products
<code>F()</code>	Use field in DB update	Increment counter
Raw SQL	Custom SQL in ORM	Complex join, DB-specific features
Signals	React to model events	Send email on user registration
Custom User Model	Extend <code>AbstractUser</code>	Add <code>phone_number</code> , make <code>email</code> unique
File Uploads	<code>FileField</code> , <code>ImageField</code>	Upload documents/images securely
Middleware	Request/response pre- and post-processing	Logging, throttling, CORS headers

 You're Now Ready for Production

These advanced features make Django **flexible, performant, and production-ready**.

If you want, I can walk you through:

- Writing complex annotated queries
- Creating a real custom user signup with phone verification
- Building a file upload system with thumbnails

Just say the word.

Deployment & Environment Setup in Django

Deploying a Django project involves **preparing your app for production** by ensuring:

- Your secrets and settings are secure
 - Static and media files are properly served
 - Your web server is robust and scalable
 - You use a production-ready database and tools
-

1. Environment Setup

Why You Need an Environment Configuration

Django projects have settings like:

- `DEBUG = True` – should be False in production
- Database credentials
- Secret keys
- Email SMTP credentials

- API keys

You **must not hardcode** these sensitive values. Instead, use **environment variables** via `.env` files.

✓ What Is a `.env` File?

A `.env` file contains environment variables in key=value format. It is **never committed to Git** and safely holds your secrets:

```
env
CopyEdit
DEBUG=False
SECRET_KEY=s0m3s3cr3tkey
ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com
DB_NAME=your_db
DB_USER=admin
DB_PASSWORD=supersecure
```

✓ Using `.env` with `python-decouple`

📦 Install:

```
bash
CopyEdit
pip install python-decouple
```

🔧 In `settings.py`:

```
python
CopyEdit
from decouple import config, Csv

DEBUG = config('DEBUG', default=False, cast=bool)
SECRET_KEY = config('SECRET_KEY')
ALLOWED_HOSTS = config('ALLOWED_HOSTS', cast=Csv())
```

✅ Using `django-environ` (alternative to `python-decouple`)

📦 Install:

bash

CopyEdit

```
pip install django-environ
```

🔧 Setup:

python

CopyEdit

```
import environ
```

```
env = environ.Env(
    DEBUG=(bool, False)
)
environ.Env.read_env()
```

```
DEBUG = env('DEBUG')
SECRET_KEY = env('SECRET_KEY')
ALLOWED_HOSTS = env.list('ALLOWED_HOSTS')
```

Both tools let you load values securely from `.env` files.

✅ Production Settings Checklist

Setting	Value in Development	Value in Production
DEBUG	True	False
ALLOWED_HOSTS	[] or localhost	Actual domain or IP
SECRET_KEY	Dummy key	Strong, secure secret key
DATABASES	SQLite	PostgreSQL/MySQL
STATIC_ROOT	Not required	Required for <code>collectstatic</code>
MEDIA_ROOT	Optional	Required if handling uploads

<code>CSRF_TRUSTED_ORIG</code>	Not needed	Required for HTTPS deployments
<code>INS</code>		

<code>SECURE_*</code> settings	Off	Enabled for HTTPS only
--------------------------------	-----	------------------------

2. Static Files in Production

Static Files Recap

- Static files: JS, CSS, images used by templates
 - During development: served via `runserver`
 - In production: you **must collect and serve them manually**
-

Step 1: Configure in `settings.py`

```
python
CopyEdit
STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles' # For collectstatic

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Step 2: Run `collectstatic`

```
bash
CopyEdit
python manage.py collectstatic
```

This gathers all `app/static` and `project/static` content into `STATIC_ROOT`, ready to be served by Nginx or another web server.

Serving Static/Media Files

In production:

- **Static files:** served directly by Nginx or the platform's web server
- **Media files** (uploads): served via media root path and mapped route

For example, with Nginx:

```
nginx
CopyEdit
location /static/ {
    alias /path/to/project/staticfiles/;
}
location /media/ {
    alias /path/to/project/media/;
}
```

3. Deployment Platforms

Let's go over **three main types** of deployments — from beginner to pro level:

A. PythonAnywhere (Beginner-Friendly)

- Shared hosting for Python apps
- Free tier supports small Django projects
- No root access, limited scalability

Steps:

1. Sign up at pythonanywhere.com
2. Upload your code or connect to GitHub
3. Go to **Web tab** → **Add a new web app**

4. Select Django, Python version
5. Enter project path and `wsgi.py` file
6. Set environment variables in **Web** → **Environment Variables**

Run:

```
bash
CopyEdit
python manage.py collectstatic
python manage.py migrate
```

7.

No need to manage Gunicorn or

No need to manage Gunicorn or Nginx directly — they're abstracted.

✓ B. Render / Railway / Heroku (Intermediate)

These platforms are **PaaS (Platform-as-a-Service)** and give you:

- Automatic deployment from GitHub
 - PostgreSQL databases
 - Domain, HTTPS, environment variable UI
 - Basic server and build setup
-

☀ Example: Render

1. Create a new web service
2. Connect to GitHub and pick repo

Add Build Command:

CSS

CopyEdit

```
pip install -r requirements.txt && python manage.py collectstatic  
--noinput && python manage.py migrate
```

- 3.
4. Set Environment Variables:

- `DEBUG=False`
- `SECRET_KEY=...`
- `DATABASE_URL=...`

Render auto-installs `gunicorn` and serves your app.

☀ Example: Railway

1. Create new Django project
2. Add PostgreSQL plugin
3. Deploy with GitHub
4. Railway auto-creates:
 - `DATABASE_URL`
 - HTTPS domain

✓ Great for MVPs, students, hackathons.

☀ Example: Heroku

1. Install Heroku CLI
2. Create a `Procfile`:

```
profile
CopyEdit
web: gunicorn myproject.wsgi
```

3. Set up:

```
bash
CopyEdit
heroku create
git push heroku main
heroku run python manage.py migrate
```

✅ C. VPS Deployment (Gunicorn + Nginx + PostgreSQL)

This is **production-grade**, used for real-world apps.

⚙️ Tools Involved

Tool	Purpose
Gunicorn	WSGI application server
Nginx	Reverse proxy, static/media server
PostgreSQL	Production-grade DB
Supervisor	Manage background processes
Docker (<i>optional</i>)	Containerization

🔧 Setup Steps

1. SSH into your VPS
2. Clone your Django repo
3. Create virtualenv, install deps

4. Setup PostgreSQL and configure in `settings.py`
5. Run `collectstatic` and `migrate`

Start Gunicorn:

```
bash
CopyEdit
gunicorn myproject.wsgi:application --bind 127.0.0.1:8000
```

- 6.

Install and configure Nginx:

```
nginx
CopyEdit
server {
    listen 80;
    server_name yourdomain.com;

    location /static/ {
        alias /home/user/project/staticfiles/;
    }

    location / {
        proxy_pass http://127.0.0.1:8000;
        include proxy_params;
    }
}
```

- 7.

Enable the site and restart Nginx:

```
bash
CopyEdit
sudo ln -s /etc/nginx/sites-available/yourproject
/etc/nginx/sites-enabled/
sudo systemctl restart nginx
```

- 8.

Use **Let's Encrypt** for free HTTPS:

```
bash
```

```
CopyEdit
```

```
sudo apt install certbot python3-certbot-nginx
```

```
sudo certbot --nginx -d yourdomain.com
```

9.

Summary Table

Feature	PythonAnywhere	Render / Heroku / Railway	VPS (Gunicorn + Nginx)
Setup Difficulty	Easy	Medium	Hard
Customization	Limited	Moderate	Full control
Free Tier	Yes	Yes	No (VPS costs money)
HTTPS Support	Yes (auto)	Yes (auto)	Yes (via Let's Encrypt)
Static/Media Handling	Handled for you	Semi-automatic	Manual setup via Nginx
Recommended For	Students, demos	Production MVPs	Real-world applications

Final Thoughts

You've now mastered **everything needed for secure and professional Django deployment**.

✓ Use `.env` + `python-decouple` or `django-environ`

✓ Use `collectstatic` properly and serve via Nginx

✓ Pick the right deployment stack for your needs

If you'd like, I can:

- Walk you through creating a complete VPS setup with Docker
- Help you deploy your project on Heroku or Render step-by-step

Just say the word!

Here is an **extremely detailed, book-level explanation** of **Django REST Framework (DRF)** topics including **serializers, viewsets, routers, authentication (token & JWT), nested serializers, throttling, pagination, permissions, and custom views** — written so thoroughly that you **do not need any other book or resource**.

REST APIs with Django REST Framework (DRF)

Overview

Django REST Framework (DRF) is a powerful toolkit for building **Web APIs** using Django. It abstracts away the complexities of request parsing, serialization, validation, and authentication while still giving full control.

A. DRF Basics

1. Serializers

Serializers convert **Django models (Python objects)** ↔ **JSON/XML (API-friendly data)**.

They also perform:

- Field-level validation
- Nested object handling
- Data transformation

✨ Two Ways to Write Serializers:

✅ **Serializer (Manual):**

python

CopyEdit

```
from rest_framework import serializers
```

```
class UserSerializer(serializers.Serializer):
```

```
username = serializers.CharField(max_length=100)
email = serializers.EmailField()
is_active = serializers.BooleanField()
```

You must implement `create()` and `update()` methods manually.

✓ **ModelSerializer (Automatic):**

python

CopyEdit

```
class UserModelSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'is_active']
```

DRF automatically:

- Maps model fields
- Implements create/update
- Adds validations

🧠 **Serializer Lifecycle**

1. Incoming request:

- `serializer = MySerializer(data=request.data)`
- `serializer.is_valid()` → checks validity
- `serializer.save()` → creates/updates instance

2. Outgoing response:

- `serializer = MySerializer(instance)`
- `serializer.data` → returns JSON-ready dict

2. Views and ViewSets

DRF Views

- APIView: DRF's version of Django's `View`, with parsing, authentication, and response handling.

```
python
CopyEdit
from rest_framework.views import APIView
from rest_framework.response import Response

class HelloAPIView(APIView):
    def get(self, request):
        return Response({'message': 'Hello, world!'})
```

ViewSet (CRUD abstraction)

- Combines logic for list/create/retrieve/update/delete.

```
python
CopyEdit
from rest_framework import viewsets
from .models import Post
from .serializers import PostSerializer

class PostViewSet(viewsets.ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
```

DRF automatically generates the logic for:

- `list()` (GET /posts/)
- `retrieve()` (GET /posts/1/)

- `create()` (POST /posts/)
 - `update()` (PUT/PATCH /posts/1/)
 - `destroy()` (DELETE /posts/1/)
-

3. Routers

Routers generate URL patterns for ViewSets automatically.

```
python
CopyEdit
from rest_framework.routers import DefaultRouter
from .views import PostViewSet

router = DefaultRouter()
router.register(r'posts', PostViewSet)

urlpatterns = [
    path('api/', include(router.urls)),
]
```

Now you have a full CRUD API at `/api/posts/`.

B. DRF Authentication

1. Token-based Authentication

DRF provides a token system:

- Users obtain a token via login.
- Clients must send `Authorization: Token <token>` header.

 **Install:**

bash
CopyEdit
`pip install djangorestframework`

 Add to `settings.py`:

python
CopyEdit
`INSTALLED_APPS += ['rest_framework.authtoken']`

`REST_FRAMEWORK = {`
 `'DEFAULT_AUTHENTICATION_CLASSES': [`
 `'rest_framework.authentication.TokenAuthentication',`
 `]`
`}`

 Migrate and Generate Token:

bash
CopyEdit
`python manage.py migrate`
`python manage.py drf_create_token <username>`

Or expose it via view:

python
CopyEdit
`from rest_framework.authtoken.views import obtain_auth_token`

`urlpatterns += [`
 `path('api/token/', obtain_auth_token),`
`]`

 2. JWT (JSON Web Tokens)

JWT is more secure and stateless. Recommended for modern applications.

 Install:

bash
CopyEdit

```
pip install djangorestframework-simplejwt
```

settings.py

python

CopyEdit

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    )
}
```

Add URLs:

python

CopyEdit

```
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns += [
    path('api/token/', TokenObtainPairView.as_view()),
    path('api/token/refresh/', TokenRefreshView.as_view()),
]
```

Client Usage:

http

CopyEdit

POST /api/token/

```
{ "username": "john", "password": "doe" }
```

Response:

```
{ "access": "...", "refresh": "..." }
```

Send Authorization:

Authorization: Bearer <access_token>

C. DRF Advanced Features

1. Nested Serializers

Used when related objects need to be included.

Example: Post with Author details

python

CopyEdit

```
class AuthorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Author
        fields = ['name', 'email']

class PostSerializer(serializers.ModelSerializer):
    author = AuthorSerializer()

    class Meta:
        model = Post
        fields = ['title', 'content', 'author']
```

If you want to allow writing nested objects, override `create()` and `update()` manually.

2. Throttling

Controls **rate of requests** per user or IP.

Types:

- `AnonRateThrottle`: limits anonymous users
- `UserRateThrottle`: limits authenticated users
- `ScopedRateThrottle`: fine-grained per-view throttle

Example:

python

CopyEdit

```
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.UserRateThrottle',
        'rest_framework.throttling.AnonRateThrottle'
    ],
    'DEFAULT_THROTTLE_RATES': {
        'user': '1000/day',
        'anon': '100/day'
    }
}
```

3. Pagination

Breaks up large responses into pages.

✓ Enable in **settings.py**:

python

CopyEdit

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10
}
```

Now API returns:

json

CopyEdit

```
{
    "count": 100,
    "next": "url?page=2",
    "previous": null,
    "results": [...]
}
```

Other types: [LimitOffsetPagination](#), [CursorPagination](#).

4. Permissions

Controls who can access what.

Built-in:

python

CopyEdit

```
from rest_framework.permissions import IsAuthenticated, IsAdminUser
```

```
class MyView(APIView):  
    permission_classes = [IsAuthenticated]
```

Custom Permission:

python

CopyEdit

```
from rest_framework.permissions import BasePermission
```

```
class IsOwner(BasePermission):  
    def has_object_permission(self, request, view, obj):  
        return obj.owner == request.user
```

Use in `ViewSet`:

python

CopyEdit

```
class PostViewSet(ModelViewSet):  
    permission_classes = [IsOwner]
```

5. Custom Views

Use `APIView` when you need full control.

python

CopyEdit

```
class CustomAPIView(APIView):  
    def get(self, request):  
        return Response({'data': 'GET success'})  
  
    def post(self, request):
```

```
serializer = MySerializer(data=request.data)
if serializer.is_valid():
    return Response(serializer.data)
return Response(serializer.errors, status=400)
```

You can also use `GenericAPIView` + mixins:

```
python
CopyEdit
class PostListCreate(generics.ListCreateAPIView):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
```

Summary Table

Feature	Purpose	Common Class
Serializer	Converts model ↔ JSON	<code>Serializer</code> , <code>ModelSerializer</code>
ViewSet	Auto-generates all CRUD views	<code>ModelViewSet</code>
Router	Auto-generates URLs	<code>DefaultRouter</code>
Auth (Token, JWT)	Secure API with token headers	DRF Token / SimpleJWT
Nested Serializer	Include related object info	Inside <code>ModelSerializer</code>
Throttling	Limit request rate	<code>UserRateThrottle</code>
Pagination	Paginate large result sets	<code>PageNumberPagination</code>
Permissions	Access control	<code>IsAuthenticated</code> , custom
Custom Views	Complete control	<code>APIView</code> , <code>GenericAPIView</code>

Ready to Build APIs

You now know:

- How to serialize data
- Use ViewSets and Routers
- Add Token and JWT auth
- Nest complex models
- Throttle users, paginate data, secure your views

This is everything you need to **build powerful, secure APIs with DRF**. If you want, I can help you:

- Build a complete DRF project step-by-step
- Add filtering, searching, or Swagger/OpenAPI docs
- Connect this API to a React/Vue frontend

Just ask, and I'll take you all the way!

40

K. Optional Extras in Django

THIRD-PARTY DJANGO PACKAGES

Django is powerful on its own, but the community has created some exceptional packages to extend its capabilities. Let's explore some of the most widely-used and production-grade ones:

1. **django-allauth** (Authentication for Humans)

django-allauth is a complete user authentication and registration system, especially useful when:

- You want **email-based signup/login**
- You want **Google, Facebook, GitHub** OAuth login

- You need **account verification, email confirmation, password reset**

Install:

bash

CopyEdit

```
pip install django-allauth
```

settings.py

python

CopyEdit

```
INSTALLED_APPS = [  
    ...  
    'django.contrib.sites',    # Required  
    'allauth',  
    'allauth.account',  
    'allauth.socialaccount',  
    'allauth.socialaccount.providers.google', # for Google login  
]
```

```
SITE_ID = 1
```

```
AUTHENTICATION_BACKENDS = [  
    'django.contrib.auth.backends.ModelBackend',  
    'allauth.account.auth_backends.AuthenticationBackend',  
]
```

```
LOGIN_REDIRECT_URL = '/'  
ACCOUNT_EMAIL_REQUIRED = True  
ACCOUNT_EMAIL_VERIFICATION = 'mandatory'  
ACCOUNT_USERNAME_REQUIRED = False
```

URLs

python

CopyEdit

```
from django.urls import path, include  
  
urlpatterns = [  
    path('accounts/', include('allauth.urls')),
```

]

This gives you out-of-the-box:

- Login/logout/register/reset
 - Email verification flows
 - OAuth integrations
 - Secure and customizable templates
-

2. **django-crispy-forms** (Better Form Rendering)

This package lets you style forms using **Bootstrap/Tailwind/etc.** with minimal code.

Install:

bash

CopyEdit

```
pip install django-crispy-forms
```

settings.py

python

CopyEdit

```
INSTALLED_APPS += ['crispy_forms']
CRISPY_ALLOWED_TEMPLATE_PACKS = "bootstrap5"
CRISPY_TEMPLATE_PACK = "bootstrap5"
```

Usage in Template

html

CopyEdit

```
{% load crispy_forms_tags %}
<form method="POST">
  {{ form|crispy }}
</form>
```

No need to manually write HTML for each field. Just install Bootstrap and this will auto-style the form.

3. **django-filter** (Query Filtering)

Enables powerful filtering of queryset results directly from the URL parameters.

Install:

bash

CopyEdit

```
pip install django-filter
```

settings.py

python

CopyEdit

```
REST_FRAMEWORK = {
    'DEFAULT_FILTER_BACKENDS':
    ['django_filters.rest_framework.DjangoFilterBackend']
}
```

In ViewSet

python

CopyEdit

```
from django_filters.rest_framework import DjangoFilterBackend
```

```
class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    filter_backends = [DjangoFilterBackend]
    filterset_fields = ['category', 'price']
```

Now you can access:

bash

CopyEdit

```
/api/products/?category=electronics&price__lte=1000
```

You can also write **custom filters** using `FilterSet` classes.

4. `django-extensions` (Developer Toolkit)

Contains dozens of tools: `shell_plus`, model graphing, UUID fields, etc.

Install:

bash

CopyEdit

```
pip install django-extensions
```

settings.py

python

CopyEdit

```
INSTALLED_APPS += ['django_extensions']
```

Usage:

- `python manage.py shell_plus` (auto-imports models)
- `python manage.py show_urls`
- `python manage.py graph_models -a -o my_models.png` (requires Graphviz)

5. Celery (Background Tasks)

Celery allows Django to offload long tasks (emails, image processing, scraping) **asynchronously** using workers.

Celery needs a message broker like **Redis or RabbitMQ**.

Install

bash

CopyEdit

```
pip install celery redis
```

Create `celery.py`

python

CopyEdit

```
# project/celery.py
```

```
import os
```

```
from celery import Celery
```

```
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "myproject.settings")
```

```
app = Celery("myproject")
```

```
app.config_from_object("django.conf:settings", namespace="CELERY")
```

```
app.autodiscover_tasks()
```

In `__init__.py`

python

CopyEdit

```
from .celery import app as celery_app
```

```
__all__ = ('celery_app',)
```

In your app:

python

CopyEdit

```
# tasks.py
```

```
from celery import shared_task
```

```
@shared_task
```

```
def send_email_task():
```

```
    # your email logic here
```

```
    ...
```

Now you can trigger `send_email_task.delay()` without blocking the user.



6. Django Channels (WebSockets + Async)



ASYNCHRONOUS DJANGO

Django 3.1+ supports `async def` views. However:

Django is synchronous by default. Use `async` only when you're integrating **I/O-heavy non-Django services** like APIs, websockets, or database pools.

✓ Writing Async Views

python

CopyEdit

```
from django.http import JsonResponse
import httpx

async def async_view(request):
    async with httpx.AsyncClient() as client:
        response = await client.get("https://httpbin.org/get")
    return JsonResponse(response.json())
```

Limitations:

- ORM is not `async`-safe yet. Avoid using `await User.objects.all()`
- Use `async` only for HTTP requests, external APIs, etc.

REAL-TIME & WEBSOCKETS

Django Channels

Django Channels adds **WebSocket support** and **asynchronous consumers**.

Use-cases:

- Real-time chat
- Live notifications
- Real-time dashboards
- Multiplayer games

Install:

```
bash
CopyEdit
pip install channels
```

settings.py

```
python
CopyEdit
INSTALLED_APPS += ['channels']
ASGI_APPLICATION = 'myproject.asgi.application'
```

asgi.py

```
python
CopyEdit
import os
from channels.routing import ProtocolTypeRouter, URLRouter
from django.core.asgi import get_asgi_application
from channels.auth import AuthMiddlewareStack
import myapp.routing

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myproject.settings')

application = ProtocolTypeRouter({
    "http": get_asgi_application(),
    "websocket": AuthMiddlewareStack(
        URLRouter(myapp.routing.websocket_urlpatterns)
    ),
})
```

routing.py (inside app)

```
python
CopyEdit
from django.urls import re_path
from . import consumers

websocket_urlpatterns = [
```

```
        re_path(r'ws/chat/(?P<room_name>\w+)/$',
consumers.ChatConsumer.as_asgi()),
]
```

WebSocket Consumer

python

CopyEdit

```
from channels.generic.websocket import AsyncWebsocketConsumer
import json
```

```
class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.room_name =
self.scope['url_route']['kwargs']['room_name']
        self.room_group_name = f'chat_{self.room_name}'

        await self.channel_layer.group_add(
            self.room_group_name, self.channel_name
        )
        await self.accept()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(
            self.room_group_name, self.channel_name
        )

    async def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json['message']

        # Broadcast to group
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'chat_message',
                'message': message
            }
        )
```

```
async def chat_message(self, event):
    await self.send(text_data=json.dumps({
        'message': event['message']
    })))
```

Channel Layers (Redis backend)

bash

CopyEdit

```
pip install channels_redis
```

python

CopyEdit

```
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [("127.0.0.1", 6379)],
        },
    },
}
```

Use Case: Real-Time Chat

1. WebSocket URL: `ws/chat/room1/`
2. User A sends a message.
3. Channels receives it via `receive()`
4. It broadcasts to all users in the same group.
5. Each client gets real-time updates.

Summary

Feature	Description	Use-case
<code>django-allauth</code>	Full user auth + OAuth	Social login, email confirmation
<code>django-crispy-forms</code>	Form rendering	Bootstrap-styled forms
<code>django-filter</code>	Query parameter filtering	REST API filtering
<code>django-extensions</code>	Dev tools	<code>shell_plus</code> , <code>graph_models</code>
<code>Celery</code>	Background processing	Emails, long tasks
<code>Async Views</code>	I/O async processing	Async API calls
<code>Channels</code>	WebSocket & real-time support	Chat apps, notifications

If you'd like, I can walk you through building a **complete real-time chat app with Channels**, **integrate Celery for async notifications**, or **build a REST API with django-filter and allauth** — just ask.

You're now equipped to go beyond traditional Django into the **real-time, async, production-ready world**.

40

Tools

ChatGPT can make mistakes. Check important info. See Cookie Preferences.